Water Level Pipeline

Introduction

This vignette walks through the complete water level processing pipeline using the `waterlogged` package. We use real test data from the ALBREDA RIVER station (ALBR_ST_30) as our example.

The pipeline processes raw HOBO U20 pressure logger data through these steps:

1. Read raw pressure logger files and link them to deployment metadata
2. Quality-check the barometric (air pressure) data
3. Read raw water level logger files
4. Attach the nearest barometric data to the water level data
5. Convert pressure readings (kPa) into water level (metres)
6. Run automatic quality checks on the water level data
7. Review plots and make manual corrections (ice, dry sensor, spikes)
8. Export the final clean dataset

Why barometric data matters: HOBO U20 loggers measure absolute pressure — that includes both the weight of water above the sensor and the weight of the atmosphere. To get just the water depth, we need to subtract the atmospheric pressure. That's why we process barometric data first.

This vignette assumes you have already installed `waterlogged` from the develop branch:

```
install.packages("pak")
pak::pkg_install("dfo-pacific-science/waterlogged@develop")
```

Setup

```
library(waterlogged)
```

Define paths to the test data shipped with the package.

```
metadata_file   <- system.file("extdata/water_level/metadata.csv", package = "waterlogged")
baro_folder     <- system.file("extdata/water_level/baro", package = "waterlogged")
level_folder    <- system.file("extdata/water_level/level", package = "waterlogged")
reference_file  <- system.file("extdata/water_level/stage_readings.csv", package = "waterlogged")
level_runs_file <- system.file("extdata/water_level/benchmarks.csv", package = "waterlogged")
```

Step 1: Bind Raw Barometric Logger Files (v0.1)

Reads all raw HOBO CSV files from the baro folder, parses them, and links each reading to its deployment in the metadata.

Raw HOBO files are individual downloads — one CSV per field visit. This step combines them into a single tidy data frame with consistent column names and deployment info attached.

How it works:

- Reads the HOBO header (row 2) to extract the serial number (SN)
- Matches the SN + timestamp window to the metadata deployments
- Renames columns to standard names (e.g., "Abs Pres kPa" → "airpress_kPa")
- Removes duplicate rows and trims data to deployment windows

Returns a list with two items: `[[1]]` = data frame of all combined readings, `[[2]]` = a quick-look plot.

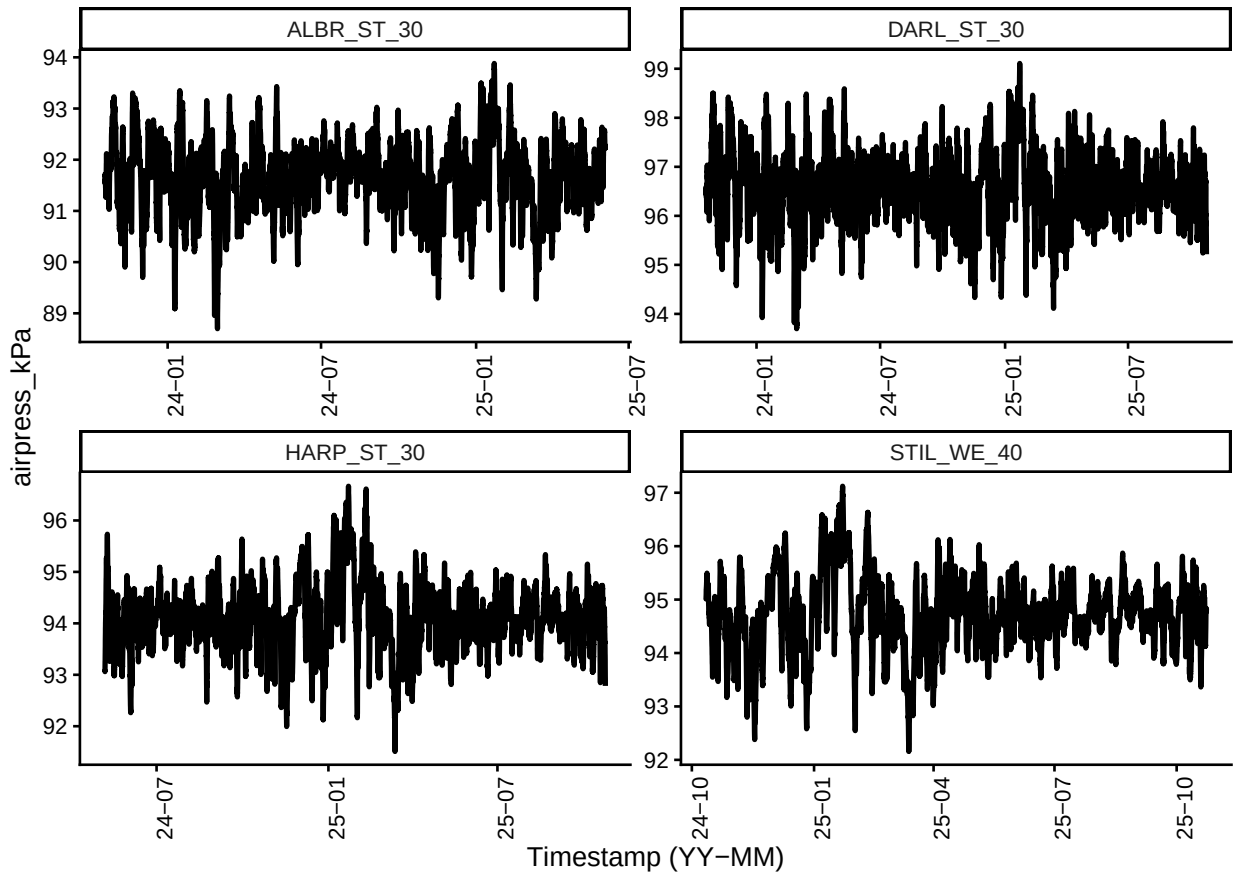
```
baro_bound <- bind_hobo_files(
  path_to_raw_folder = baro_folder,
  metadata_path      = metadata_file
)

baro_data <- baro_bound[[1]]
baro_plot <- baro_bound[[2]]

cat("Baro data dimensions:", nrow(baro_data), "rows x", ncol(baro_data), "columns\n")
#> Baro data dimensions: 53840 rows x 13 columns
cat("Stations found:", paste(unique(baro_data$site_station_code), collapse = ", "), "\n")
#> Stations found: ALBR_ST_30, DARL_ST_30, HARP_ST_30, STIL_WE_40
cat("Date range:", as.character(min(baro_data$timestamp)),
    "to", as.character(max(baro_data$timestamp)), "\n")
#> Date range: 2023-10-18 14:00:00 to 2025-10-24 13:00:00
```

```
head(baro_data)
#> # A tibble: 6 x 13
#>   sn    timestamp           airpress_kPa airtemp_C file_name timezone data1_type
#>   <chr> <dttm>                     <dbl>     <dbl> <chr>     <chr>    <chr>
#> 1 2081~ 2023-10-18 14:00:00         91.7      7.48 BARO_ALB~ GMT-07:~ Abs Pres
#> 2 2081~ 2023-10-18 15:00:00         91.7      7.68 BARO_ALB~ GMT-07:~ Abs Pres
#> 3 2081~ 2023-10-18 16:00:00         91.7      7.68 BARO_ALB~ GMT-07:~ Abs Pres
#> 4 2081~ 2023-10-18 17:00:00         91.6      7.78 BARO_ALB~ GMT-07:~ Abs Pres
#> 5 2081~ 2023-10-18 18:00:00         91.6      7.58 BARO_ALB~ GMT-07:~ Abs Pres
#> 6 2081~ 2023-10-18 19:00:00         91.6      7.48 BARO_ALB~ GMT-07:~ Abs Pres
#> # i 6 more variables: data1_unit <chr>, data2_type <chr>, dat2_unit <chr>,
#> #   site_station_code <chr>, logger_type <chr>, metric <chr>
```

```
baro_plot
```



Step 2: Quality-Check All Barometric Data

Runs automatic quality checks on the barometric data and flags problematic readings.

Barometric data feeds into every water level and dissolved oxygen calculation. If the baro data has errors (spikes, flatlines, or out-of-range values), those errors will propagate into the final water levels. Cleaning baro first protects everything downstream.

What it checks:

- Pressure too high (> 105 kPa) or too low (< 85 kPa)
- Temperature outside plausible range (-40 to 50 °C)
- Sudden spikes (> 1.5 kPa jump between consecutive readings)
- Flatlines (6 or more identical readings in a row)

The function never modifies the raw data. Instead, it creates new `_adj` (adjusted) columns that start as copies of the raw values. Any values that fail a check are set to `NA` in the `_adj` column. Boolean `edit_*` columns record exactly what was changed.

```
baro_qc <- barometric_qaqc_all(
  baro_data         = baro_data,
  metadata_path     = metadata_file,
  pressure_low_kpa  = 85,
  pressure_high_kpa = 105,
```

```
  temp_low_limit    = -40,
  temp_high_limit   = 50,
  spike_threshold_kpa = 1.5,
  flatline_n        = 6
)

cat("Baro QAQC results:\n")
#> Baro QAQC results:
cat("  Total rows:      ", nrow(baro_qc), "\n")
#>   Total rows:       53840
cat("  Pressure high:   ", sum(baro_qc$edit_press_high, na.rm = TRUE), "flagged\n")
#>   Pressure high:    0 flagged
cat("  Pressure low:    ", sum(baro_qc$edit_press_low, na.rm = TRUE), "flagged\n")
#>   Pressure low:     0 flagged
cat("  Temp out of range:", sum(baro_qc$edit_temp_range, na.rm = TRUE), "flagged\n")
#>   Temp out of range: 0 flagged
cat("  Spikes:          ", sum(baro_qc$edit_spike, na.rm = TRUE), "flagged\n")
#>   Spikes:           0 flagged
cat("  Flatlines:       ", sum(baro_qc$edit_flatline, na.rm = TRUE), "flagged\n")
#>   Flatlines:        0 flagged
cat("  Raw NAs in airpress_kPa:     ",
    sum(is.na(baro_qc$airpress_kPa)), "\n")
#>   Raw NAs in airpress_kPa:      0
cat("  Adjusted NAs in airpress_kPa_adj:",
    sum(is.na(baro_qc$airpress_kPa_adj)), "\n")
#>   Adjusted NAs in airpress_kPa_adj: 0
```

Step 3: Bind Raw Water Level Logger Files (v0.1)

Same as Step 1, but for the water level loggers instead of baro. The ALBR_ST_30 water level logger (HOBO U20, SN 20533550) was downloaded twice — once in April 2025 and once in October 2025. This step combines both downloads into one continuous record.

The water level logger measures the same physical quantity as the baro logger (absolute pressure in kPa), but because it's submerged, its readings include the pressure of the water column above it. At this stage, the column is named `waterpress_kPa` to distinguish it from the baro logger's `airpress_kPa`.

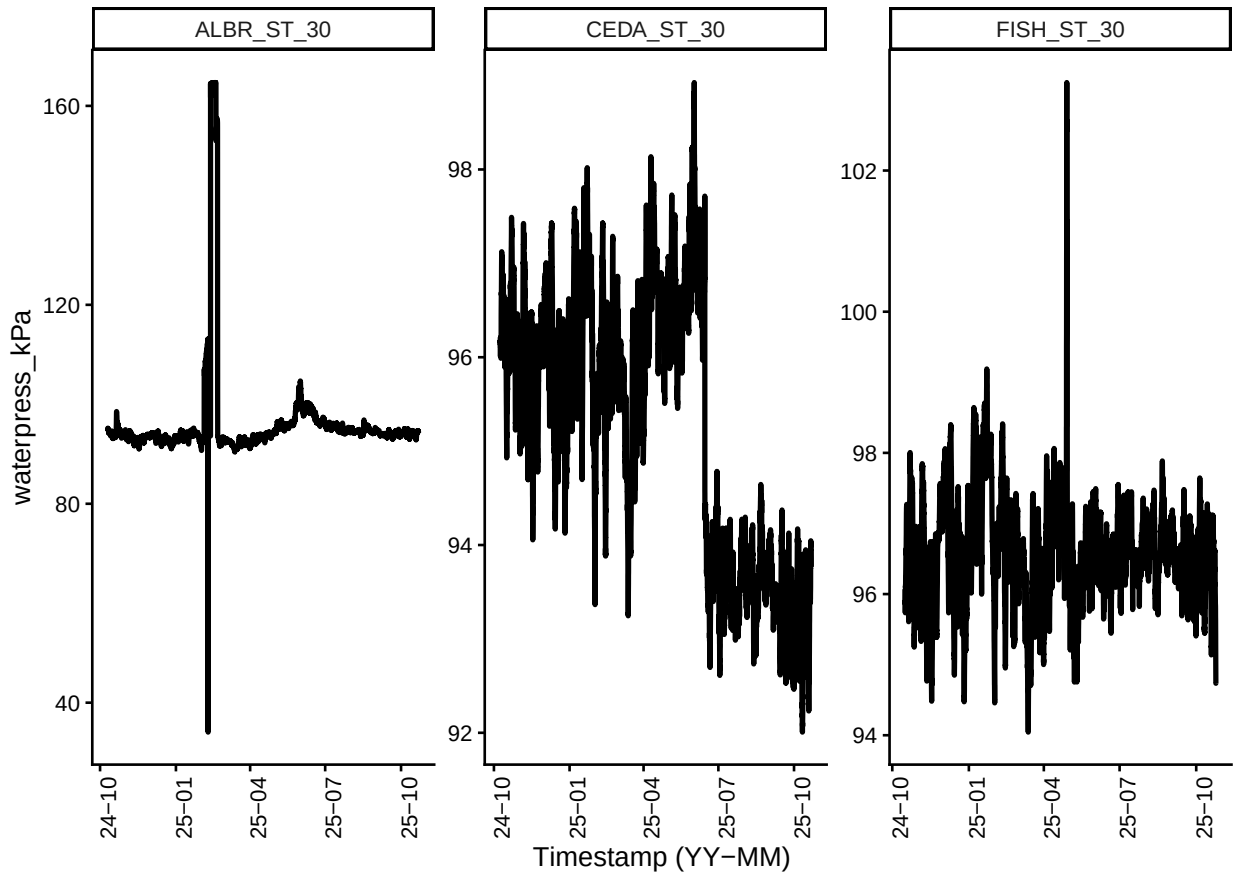
```
level_bound <- bind_hobo_files(
  path_to_raw_folder = level_folder,
  metadata_path      = metadata_file
)

level_data <- level_bound[[1]]
level_plot <- level_bound[[2]]

cat("Water level data dimensions:", nrow(level_data), "rows x",
    ncol(level_data), "columns\n")
#> Water level data dimensions: 27080 rows x 13 columns
cat("Stations found:", paste(unique(level_data$site_station_code),
    collapse = ", "), "\n")
#> Stations found: ALBR_ST_30, CEDA_ST_30, FISH_ST_30
```

```
head(level_data)
#> # A tibble: 6 x 13
#>   sn       timestamp           waterpress_kPa watertemp_C file_name     timezone
#>   <chr>    <dttm>                       <dbl>       <dbl> <chr>         <chr>
#> 1 20533550 2024-10-10 10:00:00           95.1        5.76 WL_ALBR_ST_3~ GMT-07:~
#> 2 20533550 2024-10-10 11:00:00           95.0        5.86 WL_ALBR_ST_3~ GMT-07:~
#> 3 20533550 2024-10-10 12:00:00           95.2        6.06 WL_ALBR_ST_3~ GMT-07:~
#> 4 20533550 2024-10-10 13:00:00           95.1        6.27 WL_ALBR_ST_3~ GMT-07:~
#> 5 20533550 2024-10-10 14:00:00           95.0        6.37 WL_ALBR_ST_3~ GMT-07:~
#> 6 20533550 2024-10-10 15:00:00           95.1        6.57 WL_ALBR_ST_3~ GMT-07:~
#> # i 7 more variables: data1_type <chr>, data1_unit <chr>, data2_type <chr>,
#> #   dat2_unit <chr>, site_station_code <chr>, logger_type <chr>, metric <chr>
```

```
level_plot
```



Step 4: Attach Barometric Data to Water Level (v0.2)

For each water level reading, finds the nearest barometric station and attaches the matching air pressure and temperature values.

To convert pressure to water depth, we need to know the atmospheric pressure at the same time. This step pairs each water level timestamp with the closest baro reading.

How "auto" selection works:

1. Calculates the geographic distance (Haversine formula) between each water level station and every barometric station in the metadata
2. Selects up to 3 nearest baro stations (within 30 km by default)
3. If multiple baro stations overlap in time, calibrates them against the closest one and combines them to fill gaps
4. Prefers the QAQC-adjusted values (`airpress_kPa_adj`) when available

New columns added: `airpress_kPa`, `airtemp_C`, `baro_site_stn_code`.

```
wl_with_baro <- add_nearest_baro(
  input_data         = level_data,
  baro_data          = baro_qc,
  metric             = "water level",
  baro_site_selection = "auto",
  metadata_path      = metadata_file
)

wl_data     <- wl_with_baro[[1]]
wl_baro_plot <- wl_with_baro[[2]]

cat("After baro attach:", nrow(wl_data), "rows x", ncol(wl_data), "columns\n")
#> After baro attach: 27080 rows x 20 columns
cat("Baro station(s) used:",
    paste(unique(wl_data$baro_site_stn_code), collapse = ", "), "\n")
#> Baro station(s) used: ALBR_ST_30, STIL_WE_40, DARL_ST_30, HARP_ST_30, NA

baro_na_count <- sum(is.na(wl_data$airpress_kPa))
cat("Timestamps with missing baro data:", baro_na_count,
    paste0("(", round(baro_na_count / nrow(wl_data) * 100, 1), "%)"), "\n")
#> Timestamps with missing baro data: 3 (0%)
```

Note that stations are dropped if the function can't find a baro match:

```
cat("Stations in level_data:", paste(unique(level_data$site_station_code), collapse = ", "), "\n")
#> Stations in level_data: ALBR_ST_30, CEDA_ST_30, FISH_ST_30
cat("Stations in wl_data:   ", paste(unique(wl_data$site_station_code), collapse = ", "), "\n")
#> Stations in wl_data:    ALBR_ST_30, CEDA_ST_30, FISH_ST_30
```

```
wl_baro_plot
```



Step 5: Convert Pressure (kPa) to Water Level (metres) (v0.3)

Converts the raw pressure readings into actual water depth/stage in metres, using physics and a field reference measurement.

How the conversion works:

1. Calculates water density based on temperature (Kell 1975 equation)
2. Converts water pressure to a depth using: depth = pressure / (density × g)
3. Converts baro pressure to an equivalent depth the same way
4. Subtracts baro depth from water depth to get the water column height
5. Adds a constant "k" to anchor the result to a real-world reference (e.g., a staff gauge reading taken during a field visit)

The `select_measurement` parameter selects which field visit to use as the anchor point. The `reference_type` can be `"stage"` (staff gauge reading) or `"depth"` (direct depth measurement).

```
converted <- convert_waterlevel_kPa_m(
  input_data       = wl_data,
  select_station   = "ALBR_ST_30",
  reference_data   = reference_file,
  reference_type   = "stage",
  select_measurement = 1,
  return_plots     = TRUE
```

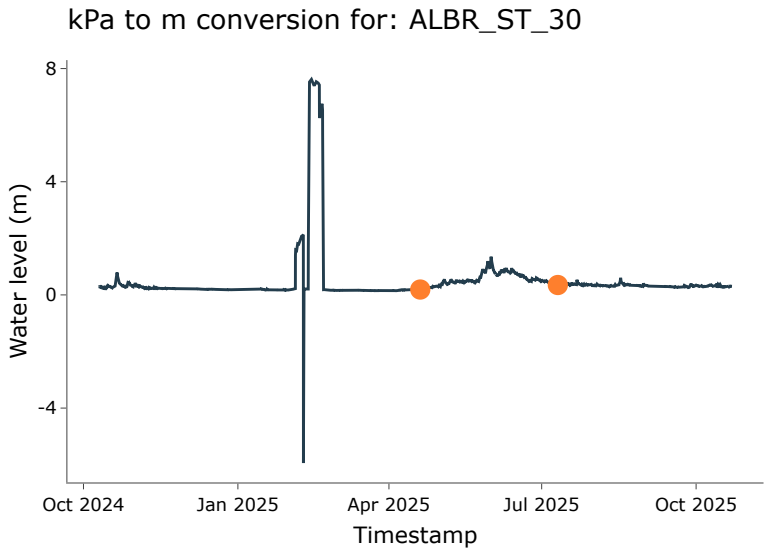
```
)

site_wl <- converted$site_wl
ref_dat <- converted$ref_dat
wl_plot <- converted$wl_plot

cat("Converted data:", nrow(site_wl), "rows\n")
#> Converted data: 9047 rows
cat("Water level range:",
    round(min(site_wl$waterlevel_m, na.rm = TRUE), 3), "to",
    round(max(site_wl$waterlevel_m, na.rm = TRUE), 3), "metres\n")
#> Water level range: -5.943 to 7.621 metres
cat("Reference measurements used:", nrow(ref_dat), "\n")
#> Reference measurements used: 2
```

The interactive plot overlays the computed water level (blue line) with the field reference points (orange dots). If the logger is well-calibrated, the dots should sit on or near the line.

```
wl_plot
```




```
head(site_wl[, c("timestamp", "waterpress_kPa", "airpress_kPa",
                 "watertemp_C", "waterlevel_m", "ref_m")])
```

```
#> # A tibble: 6 x 6
#>   timestamp           waterpress_kPa airpress_kPa watertemp_C waterlevel_m ref_m
#>   <dttm>                       <dbl>        <dbl>       <dbl>         <dbl> <dbl>
#> 1 2024-10-10 10:00:00           95.1         92.5        5.76         0.310    NA
#> 2 2024-10-10 11:00:00           95.0         92.5        5.86         0.304    NA
#> 3 2024-10-10 12:00:00           95.2         92.5        6.06         0.319    NA
#> 4 2024-10-10 13:00:00           95.1         92.5        6.27         0.309    NA
#> 5 2024-10-10 14:00:00           95.0         92.5        6.37         0.298    NA
#> 6 2024-10-10 15:00:00           95.1         92.5        6.57         0.303    NA
```

Step 6: Automatic Water Level QAQC

Runs automatic quality checks on the computed water level data and flags suspicious values. Even after a good conversion, the data may contain artifacts from ice formation, sensor drying out, or physical disturbances (e.g., a flood moving the logger).

What it checks:

- Negative water temperatures (< -1 °C removed; -1 to 0 °C set to 0)
- Ice conditions (water temp < 0.3 °C flagged but NOT removed)
- Step disturbances (water level jump > 0.1 m between readings)
- Dry sensor indicators (warm water temp + low pressure pattern)

Important distinction — "flag" vs "edit":

- `flag_*` columns = "we noticed something, please review" — the data is NOT changed.
- `edit_*` columns = "we changed the data" — the `_adj` value was modified (usually set to `NA`).

```
wl_qc <- waterlevel_qaqc(
  input_data     = site_wl,
  select_station = "ALBR_ST_30"
)

cat("QAQC results for ALBR_ST_30:\n")
#> QAQC results for ALBR_ST_30:
cat("  Total rows:        ", nrow(wl_qc), "\n")
#>   Total rows:         9047
cat("  Ice flagged:       ", sum(wl_qc$flag_ice, na.rm = TRUE), "readings\n")
#>   Ice flagged:        766 readings
cat("  Disturbance flagged:", sum(wl_qc$flag_disturbance, na.rm = TRUE), "readings\n")
#>   Disturbance flagged: 16 readings
cat("  Dry flagged:       ", sum(wl_qc$flag_wl_dry, na.rm = TRUE), "readings\n")
#>   Dry flagged:        4980 readings
cat("  Temp < -1 C removed:", sum(wl_qc$neg_lt_minus1, na.rm = TRUE), "readings\n")
#>   Temp < -1 C removed: 194 readings
cat("  Temp -1 to 0 C fixed:", sum(wl_qc$neg_between, na.rm = TRUE), "readings\n")
#>   Temp -1 to 0 C fixed: 270 readings
```

The QAQC log is stored as an attribute on the result. It records every automated action taken.

```r
qaqc_log <- attr(wl_qc, "qaqc_log")

if (!is.null(qaqc_log) && nrow(qaqc_log) > 0) {
  cat("QAQC log entries:\n")
  print(qaqc_log[, c("field", "action", "code", "ts_start", "ts_end")])
}
#> QAQC log entries:
#> # A tibble: 4 x 5
#>   field            action    code         ts_start            ts_end
#>   <chr>            <chr>     <chr>        <dttm>              <dttm>
#> 1 watertemp_C_adj  AUTO_QAQC NEGATIVE_WT  2025-01-20 05:00:00 2025-02-20 12:00:00
#> 2 watertemp_C_adj  AUTO_QAQC FLAG_ICE     2025-01-02 06:00:00 2025-02-21 14:00:00
#> 3 waterlevel_m_adj AUTO_QAQC FLAG_DISTU~ 2025-02-04 06:00:00 2025-02-20 15:00:00
#> 4 waterlevel_m_adj AUTO_QAQC FLAG_WL_DRY 2024-10-10 10:00:00 2025-10-21 20:00:00
```

Step 7: Visual Review

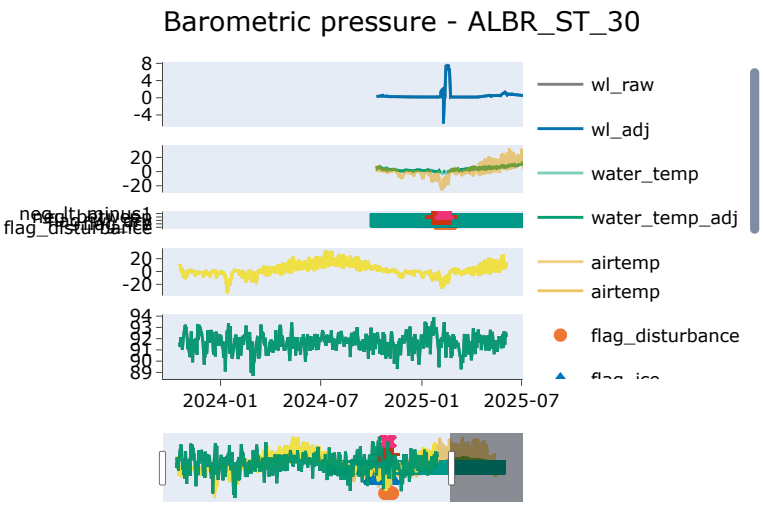
Creates an interactive multi-panel plot for reviewing the data. Before making manual corrections, you need to see the data. The plot shows raw vs adjusted values with flag indicators so you can decide what needs fixing.

How to use the plot:

- Light traces = raw values (original, unmodified)
- Dark traces = adjusted values (after QAQC corrections)
- Colored markers = flag locations (ice, dry, disturbance)
- Use the range slider at the bottom to zoom in on specific periods
- Hover over points to see exact values and timestamps

This is an iterative process: view the plot, identify a problem period, apply a correction, re-plot to verify, and repeat until satisfied.

```r
plot_qaqc_timeseries(
  wl_data        = wl_qc,
  baro_data      = baro_qc,
  select_station = "ALBR_ST_30"
)
```


Barometric pressure - ALBR_ST_30
8
4
0
-4
20
0
-20
20
0
-20
94
93
92
91
90
89
2024-01
2024-07
2025-01
2025-07
wl_raw
wl_adj
water_temp
water_temp_adj
airtemp
airtemp
flag_disturbance

Step 8: Manual Adjustments (Examples)

This section demonstrates the manual correction tools available. In a real project, you would apply these only after reviewing the plot and identifying specific problem periods.

Each function returns a new data frame — it does not modify the original. Always assign the result back to your working variable so corrections accumulate. The examples below use placeholder timestamps; in practice, you would pick these from the interactive plot.

8a. Remove ice-affected data (set to NA)

Sets `waterlevel_m_adj` and `watertemp_C_adj` to `NA` for a time window where ice was affecting the sensor. When ice forms around a pressure sensor, it creates artificial pressure readings that don't represent the actual water level. There's no way to correct for ice — the only option is to remove the affected readings entirely.

```
wl_qc <- adjust_logger_NA(
   input_data       = wl_qc,
   select_station   = "ALBR_ST_30",
   metric           = "waterlevel",
   reason_to_adjust = "ice",
   timestamp_start  = "2025-02-03 00:00:00",
   timestamp_end    = "2025-02-10 00:00:00",
   manual_note      = "Ice confirmed from field visit and temperature data"
)
```

8b. Correct a spike / flatline (linear interpolation)

Replaces `waterlevel_m_adj` values within a time window with a straight line drawn between the start and end points. Appropriate when the spike is clearly an artifact and the true water level was likely smooth through that period.

```
wl_qc <- adjust_waterlevel_spike(
  input_data       = wl_qc,
  select_station   = "ALBR_ST_30",
  timestamp_start  = "2025-02-03 21:00:00",
  timestamp_end    = "2025-02-10 16:00:00",
  reason_to_adjust = "ice",
  manual_note      = "Interpolated across ice spike"
)
```

8c. Correct a step offset (cable break / logger moved)

Applies a constant vertical shift to all data from a specific point onward, correcting for a sudden physical displacement. If the logger physically moves (e.g., a flood shifts the mounting bracket), all readings after that point are offset by a constant amount.

```
wl_qc <- adjust_WL_offset(
  input_data      = wl_qc,
  select_station  = "ALBR_ST_30",
  timestamp_start = "2025-02-04 07:00:00",
  timestamp_end   = "2025-02-09 00:00:00",
```

```
  manual_note     = "Logger displaced by high water event"
)
```

8d. Multi-point drift correction

Corrects gradual sensor drift using known offsets at specific timestamps. Applies piecewise-linear interpolation between points.

```
drift_pts <- tibble::tibble(
  timestamp = c("2024-10-10 10:00:00", "2025-06-01 00:00:00"),
  drift_m   = c(0.0, 0.12)
)

wl_qc <- wl_multipoint_correction(
  input_data     = wl_qc,
  select_station = "ALBR_ST_30",
  drift_points   = drift_pts,
  manual_note    = "Drift correction based on field survey checks"
)
```

After any manual adjustments, always re-plot to verify:

```
plot_qaqc_timeseries(
  wl_data        = wl_qc,
  baro_data      = baro_qc,
  select_station = "ALBR_ST_30"
)
```

Step 9 (Optional): Set Datum from Level Runs

Converts water level from relative stage (metres above an arbitrary reference) to absolute elevation (metres above sea level) using surveyed benchmark data.

Staff gauge readings give you relative water level ("how high is the water on the gauge?"). Level runs tie that gauge to real-world elevation using surveying equipment. This step adds a `wl_elev_m` column with the absolute elevation.

Note: This function is marked as "still in testing" by the developers. It works for initial datum establishment but has limitations for multi-year reapplication.

```
wl_datum <- wl_set_datum(
  input_data      = wl_qc,
  level_runs_path = level_runs_file,
  select_station  = "ALBR_ST_30"
)

head(wl_datum$wl_elev_m)
#> [1] 100.3948 100.3888 100.4040 100.3942 100.3836 100.3881
```

Step 10: Export Clean Data (v1.0)

Writes the final cleaned data to CSV files, one per station per year. This creates the deliverable output — the clean, quality-controlled water level record ready for analysis, reporting, or sharing.

What gets exported:

- `site_station_code`, `timestamp` (identifiers)
- `waterlevel_m` (raw, unmodified)
- `waterlevel_m_adj` (the final adjusted value — use this for analysis)
- `watertemp_C`, `watertemp_C_adj` (raw and adjusted temperature)
- All `flag_*` and `edit_*` columns (the audit trail)

Output file naming: `<output_folder>/<year>/clean/<station>_WL_<start>_<end>_v1.0.csv`

```
write_clean_data(
  input_data            = wl_qc,
  path_to_output_folder = "processed",
  metric                = "waterlevel"
)
```

Recap

Step	Function	Result
1	`bind_hobo_files()` on baro CSVs	`baro_data` (raw baro)
2	`barometric_qaqc_all()`	`baro_qc` (cleaned baro)
3	`bind_hobo_files()` on level CSVs	`level_data` (raw WL)
4	`add_nearest_baro()`	`wl_data` (WL + baro attached)
5	`convert_waterlevel_kPa_m()`	`site_wl` (kPa → metres)
6	`waterlevel_qaqc()`	`wl_qc` (auto QAQC'd)
7	`plot_qaqc_timeseries()`	(visual review)
8	Manual adjustments (as needed)	`wl_qc` (updated)
9	`wl_set_datum()` (optional)	(adds elevation)
10	`write_clean_data()`	CSV files on disk

Key columns to understand:

- `waterlevel_m` — the raw computed water level (never modified)
- `waterlevel_m_adj` — the adjusted version (all corrections applied here)
- `flag_*` — "something looks off, please review" (data unchanged)
- `edit_*` — "we changed the data" (data was set to NA or corrected)

Always use `waterlevel_m_adj` for analysis. Compare it with `waterlevel_m` to see exactly what the QAQC process changed.